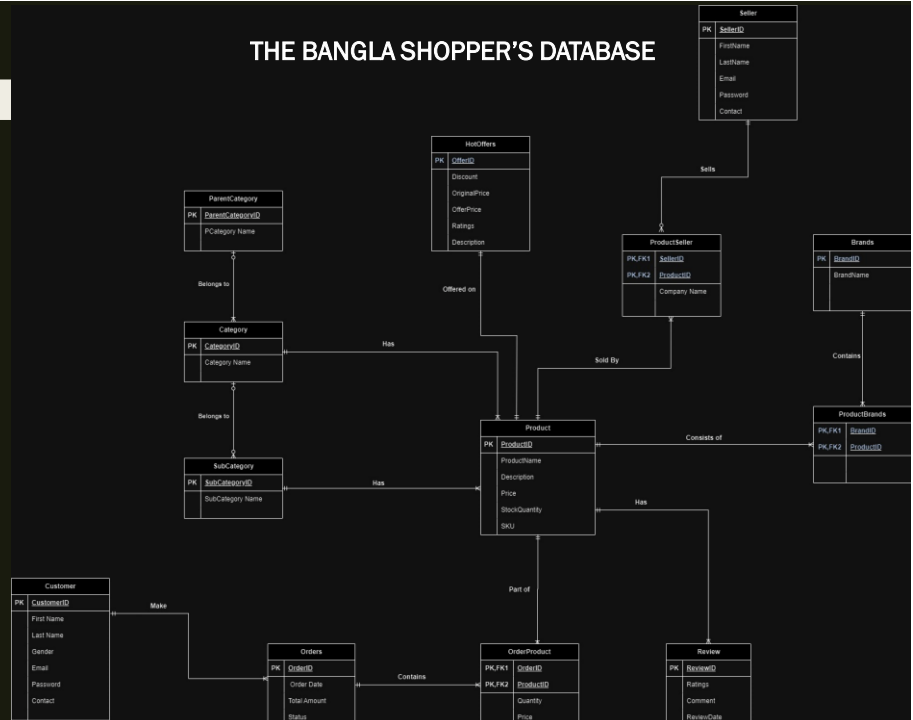


RAFIA TASNEEM
ID: 24869289

THE BANGLA SHOPPER'S DATABASE



1



HIGH DISTINCTION ASSIGNMENT- PROJECT-1

By Rafia Tasneem

2

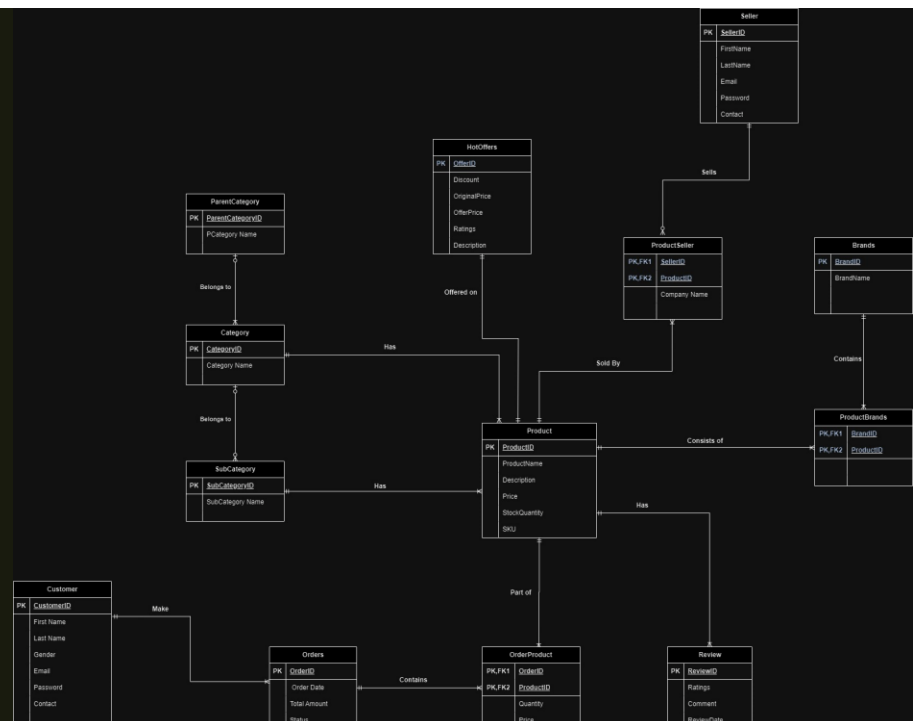
THE BANGLA SHOPPER'S DATABASE:

Bangla Shoppers is an authorized distributor and retailer of several international brands of perfumes and cosmetics in Bangladesh, established in 2007 and in business since 2013. They are an e-commerce business that sells anything from premium beauty products and fragrances to lifestyle goods, skincare products, hair care products, and some more.

I have attempted to create a simulated database of this shop using the data provided on their website (<https://www.banglashoppers.com/>). However, for security and confidentiality, data for customers, sellers, purchase and detailed transaction are not accessible on the internet. The generated database provides information about Bangla Shoppers' performance in the local marketplace.

3

Entity Relationship Diagram (ERD)

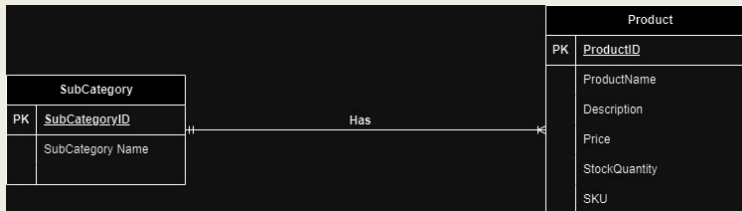


4

ERD Explanation:

One to Many Relationship:

To represent a one-to-many relationship within ERD, a line is used with an arrowhead extending from the "one" side to the "many" side. When several instances of one entity (the "one" side) are connected to numerous instances of another entity (the "many" side), this kind of connection is used.



```

postgres=# SELECT ProductName, Price FROM Product WHERE SubCategoryID=1;
              productname              | price
-----+-----
  
```

```

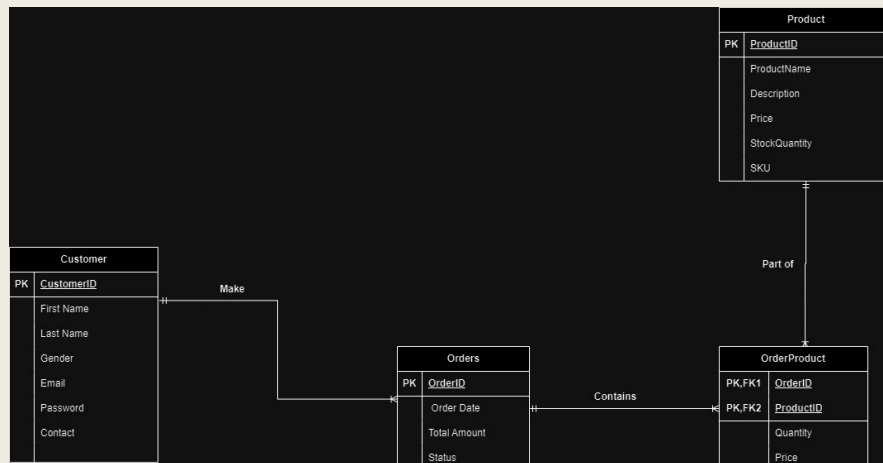
  Technic 15 Color Eye Shadow Palette - Vacay - 30g | 550.00
  Lois Chloe 10 Color Highly Pigmented Signature Eyeshadow Palette - 15gm | 4550.00
(2 rows)
  
```

5

ERD Explanation (Cont.):

Many to Many Relationship:

A connection between two entities where one entity can be related to several instances of another entity and oppositely is referred to as a "many-to-many relationship"



6

Many to Many Relationship (Cont.):

This relationship retrieves information about customers, their orders, and the products within each order. It establishes connections between the 'Customer', 'Orders', 'OrderProduct' and 'Product' tables. Each customer can place multiple orders, and each order can contain multiple products. Therefore, it represents a many-to-many relationship between customers and products, mediated by the intermediary table 'OrderProduct'.

customerid	firstname	lastname	orderid	orderdate	productid	quantity	price
1	Kevin	Reed	1	2024-04-01	1	2	550
1	Kevin	Reed	1	2024-04-01	3	1	699
2	Charlie	Smith	2	2024-04-02	5	3	290
2	Charlie	Smith	2	2024-04-02	8	1	950
3	John	Doe	3	2024-04-03	11	2	525
4	Emily	Johnson	4	2024-04-04	17	4	350
4	Emily	Johnson	4	2024-04-04	21	1	399
5	Michael	Brown	5	2024-04-05	25	2	320
5	Michael	Brown	5	2024-04-05	27	1	2250
1	Kevin	Reed	6	2024-04-06	29	1	2150

(10 rows)

7

SQL QUERIES

a) Simple Query of a Single Table:

```
SELECT * FROM ParentCategory;
```

The query `SELECT * FROM ParentCategory;` retrieves all columns and rows from the table named `ParentCategory`. The asterisk (*) symbol is a wildcard character that represents all columns in the specified table. Therefore, this query essentially fetches all data stored in the `ParentCategory` table.

parentcategoryid	parentcategory_name
1	Makeup Shop
2	Health & Beauty Shop
3	Bath & Body Shop
4	Hair Care Shop
5	Perfume Shop

(5 rows)

8

SQL QUERIES

b) Query Using 'NATURAL JOIN':

SELECT * FROM ProductBrands NATURAL JOIN Brands;

This query performs a natural join between the tables ProductBrands and Brands. A natural join is a type of join operation that combines rows from both tables based on matching column names. In this case, it joins the tables on any columns that have the same name in both tables. The result is a new table containing all columns from both tables where the corresponding columns have matching values.

brandid	productid	brandname
20	1	Technic Cosmetics
14	2	Lois Chloe Cosmetics
19	3	St. Ives
23	4	W7 Cosmetics
19	5	St. Ives
19	6	St. Ives
1	8	Absolute New York
1	9	Absolute New York
20	13	Technic Cosmetics
20	14	Technic Cosmetics
19	15	St. Ives
19	16	St. Ives
20	18	Technic Cosmetics
19	19	St. Ives
1	20	Absolute New York

(15 rows)

9

SQL QUERIES

c) The cross product equivalent to the "natural join" query:

SELECT * FROM ProductBrands, Brands WHERE ProductBrands.BrandID = Brands.BrandID;

This query retrieves all columns from the tables ProductBrands and Brands where the BrandID column in ProductBrands matches the BrandID column in Brands. It essentially performs a cross product or Cartesian join between the two tables, combining every row from ProductBrands with every row from Brands where the BrandID values are equal. Then, it filters the result to include only the rows where the BrandID values match.

brandid	productid	brandname
20	1	Technic Cosmetics
14	2	Lois Chloe Cosmetics
19	3	St. Ives
23	4	W7 Cosmetics
19	5	St. Ives
19	6	St. Ives
1	8	Absolute New York
1	9	Absolute New York
20	13	Technic Cosmetics
20	14	Technic Cosmetics
19	15	St. Ives
19	16	St. Ives
20	18	Technic Cosmetics
19	19	St. Ives
1	20	Absolute New York

(15 rows)

10

SQL QUERIES

d) A query involving a “Group by”, perhaps also with a “HAVING”:

This query retrieves the BrandID and BrandName from the Brands table, along with a count of the associated products for each brand. It joins the Brands table with the ProductBrands table based on the BrandID. Then, it groups the results by BrandID and BrandName. The HAVING clause filters the grouped results to include only those brands where the count of associated products (ProductID) is greater than 2. In other words, it selects brands that have more than 2 products associated with them.

```
SELECT b.BrandID, b.BrandName,
COUNT(pb.ProductID) AS ProductCount
FROM Brands b
JOIN ProductBrands pb ON b.BrandID =
pb.BrandID
GROUP BY b.BrandID, b.BrandName
HAVING COUNT(pb.ProductID) > 2;
```

brandid	brandname	productcount
19	St. Ives	6
20	Technic Cosmetics	4
1	Absolute New York	3

(3 rows)

11

SQL QUERIES

e) A query that uses a subquery:

```
SELECT FirstName, LastName
FROM Seller
WHERE SellerID IN (
    SELECT SellerID
    FROM ProductSeller
    WHERE CompanyName IN (
        SELECT BrandName
        FROM Brands
        WHERE BrandName = 'Cerave'
    )
);
```

firstname	lastname
Bob	Johnson
Diana	Garcia

(2 rows)

This query retrieves the first and last names of sellers who are associated with products sold by a company that distributes the brand 'Cerave'. The innermost subquery selects the BrandName from the Brands table where the brand is 'Cerave'. The middle subquery selects the SellerID from the ProductSeller table where the CompanyName matches the brand name obtained from the previous subquery. The outer query selects the FirstName and LastName from the Seller table where the SellerID matches any of the SellerIDs obtained from the middle subquery.

12

SQL QUERIES

f) A cross product which cannot be implemented using the words “natural join” (e.g. self join):

This query generates distinct pairs of seller IDs (Seller1 and Seller2) from the ProductSeller table, where each pair represents two different sellers who sell products for the same company.

```
SELECT DISTINCT ps1.SellerID AS Seller1, ps2.SellerID AS
Seller2
FROM ProductSeller ps1
CROSS JOIN ProductSeller ps2
WHERE ps1.SellerID < ps2.SellerID AND
ps1.CompanyName = ps2.CompanyName;
```

seller1	seller2
2	4
2	3
3	5
1	5
1	3
4	5
3	4
1	4

(8 rows)

13

‘CHECK’ Constraint

The CHECK constraint in SQL ensures that the values inserted or updated in a column meet specific conditions defined by a Boolean expression. This constraint adds a level of data integrity by validating the data before it is stored in the database. The condition can involve comparisons, logical operators, functions, or any valid expression that evaluates to true or false. By enforcing these conditions, CHECK constraints help maintain data accuracy and consistency in the database. The following constraints are being used in our database:

- CONSTRAINT di_table_Customer_gender CHECK (Gender IN ('M','F'))
- CONSTRAINT di_table_Customer_contact CHECK (Contact is NOT NULL)
- CONSTRAINT di_table_Orders_status CHECK (Status IN ('Processing', 'Shipped', 'Delivered'))
- CONSTRAINT di_table_Review_ratings CHECK (Ratings>=0 and Ratings<=5)

14

‘ON DELETE RESTRICT’, ‘ON UPDATE CASCADE’

ON DELETE RESTRICT:

CONSTRAINT FK_Orders FOREIGN KEY (OrderID) REFERENCES Orders ON DELETE RESTRICT ON UPDATE CASCADE:

The constraint ensures that the OrderID column in the OrderProduct table references the OrderID column in the Orders table. It specifies the following actions:

- **ON DELETE RESTRICT:** Prevents deletion of a record from the Orders table if there are dependent records in the current table. This restriction maintains referential integrity.
- **ON UPDATE CASCADE:** Automatically updates the OrderID in the current table if the corresponding OrderID in the Orders table is updated. This cascade ensures consistency in the foreign key references.

15

‘ON DELETE CASCADE’, ‘ON UPDATE CASCADE’

ON DELETE CASCADE:

CONSTRAINT FK_Product FOREIGN KEY (ProductID) REFERENCES Product ON DELETE CASCADE ON UPDATE CASCADE:

This constraint ensures that the ProductID column in the OrderProduct table references the ProductID column in the Product table. It specifies the following actions:

- **ON DELETE CASCADE:** If a record is deleted from the Product table, all dependent records in the current table are also deleted. This cascade action maintains consistency by removing related records.
- **ON UPDATE CASCADE:** If the ProductID in the Product table is updated, the corresponding ProductID values in the current table are automatically updated. This ensures that the foreign key references remain synchronized with any changes in the primary key.

16

VIEW

- A view in a database facilitates data manipulation, protects sensitive data, and streamlines complex queries. It is a useful tool for organizing and simplifying data by combining information from many tables. The following view has been used in the database:

```
CREATE VIEW ProductBrandView AS
SELECT p.ProductID, p.ProductName, b.BrandName
FROM Product AS p
JOIN ProductBrands AS pb ON p.ProductID = pb.ProductID
JOIN Brands AS b ON pb.BrandID = b.BrandID;

SELECT * FROM ProductBrandView;
```

17

VIEW (Cont.)

The following query creates a view called ProductBrandView that combines product names with their corresponding brand names. Then, it selects all data from this view, displaying the product IDs, names, and their associated brand names. Essentially, it simplifies querying by providing a consolidated and easy-to-use dataset.

productid	productname	brandname
1	Technic 15 Color Eye Shadow Palette - Vacay - 30g	Technic Cosmetics
2	Lois Chloe 10 Color Highly Pigmented Signature Eyeshadow Palette - 15gm	Lois Chloe Cosmetics
3	Wet n Wild Photo Focus Eyeshadow Primer (Only A Matter Of Prime)	St. Ives
4	W7 Liquid Eyeliner Pot 8ml - Black	W7 Cosmetics
5	Wet n Wild Color Icon Khol Liner Pencil (You Are Always White)	St. Ives
6	Wet n Wild Eye Brow Pomade (Brunette)	St. Ives
8	Absolute New York Multipack 5D Darling Eye Lashes - ELDL61 Cecilia	Absolute New York
9	Absolute New York Cashmere - Stella - 5D Zero Gravity Comfort Flat Band Eye Lashes - ELCL19	Absolute New York
13	Technic Mega Matte Blush - 11.2gm	Technic Cosmetics
14	Technic Mega Matte Bronzer - 11.2gm	Technic Cosmetics
15	Wet n Wild Prime Focus Pore Minimizing Primer	St. Ives
16	Wet N Wild Bare Focus Clarifying Finishing Powder - Fair/Light 6g	St. Ives
18	Technic Mega Glow Highlighter - 10gm	Technic Cosmetics
19	Wet n Wild Photo Focus Matte Finish Makeup Setting Spray 45ml	St. Ives
20	Absolute New York Matte Made In Heaven Liquid Lipstick & Liner Duo - MLIH03 - Hyped	Absolute New York

(15 rows)

18

THANK YOU